

Clase 2

FUNCIONES

CC1002 Introducción a la
Programación

Expresiones con variables

- Si el radio de un círculo esta dado por la variable r , su área se calcula mediante la expresión:
$$\text{areaCirculo} = 3.14r^2$$
- ¿Qué pasa si se requiere evaluar esta expresión muchas veces en el programa?
 - Mala práctica de programación: duplicar código

CC1002 Introducción a la
Programación

Funciones

- Función: estructura que recibe valores de entrada y genera un valor de salida
 - Es importante que cada función sea nombrada de forma en que sea clara la relación entre su nombre y el objetivo que cumple
 - Permite tener una implementación única de una expresión, de forma de poder reutilizarla

CC1002 Introducción a la
Programación

Funciones

- Función que calcula área del círculo

```
def areaCirculo(radio):  
    return 3.14 * radio ** 2
```

 - La instrucción **def** sirve para definir la función
 - Los argumentos van entre paréntesis
 - La instrucción **return** indica el fin de la función, evalúa la expresión y retorna el resultado

CC1002 Introducción a la
Programación

Funciones

- Para utilizar una función, debemos invocarla con el valor de sus argumentos

```
>>> areaCirculo(5)  
78.5
```

CC1002 Introducción a la
Programación

Funciones

- Problema: definir una función que calcule el área de un anillo



CC1002 Introducción a la
Programación

Funciones

- Algoritmo
 - Calcular área de círculo grande (exterior)
 - Calcular área de círculo chico (interior)
 - Área anillo: Resta de ambas áreas
- Solución (usando mala práctica de programación):

```
def areaAnillo(exterior, interior):  
    return 3.14 * exterior ** 2 - 3.14 * interior ** 2
```

CC1002 Introducción a la
Programación

Funciones

- Expresiones con funciones
- Solución (buena práctica de programación):

```
def areaAnillo(exterior, interior):  
    return areaCirculo(exterior) - areaCirculo(interior)
```

```
>>> areaAnillo(5, 3)  
50.24
```

```
areaAnillo(5, 3) → areaCirculo(5) - areaCirculo(3)  
→ 3.14 * 5 ** 2 - 3.14 * 3 ** 2  
→ 3.14 * 25 - 3.14 * 9  
→ 50.24
```

CC1002 Introducción a la
Programación

Funciones

- Código DEBE estar indentado:

```
def areaCirculo(radio):  
    pi = 3.14  
    return pi * radio * radio
```

CC1002 Introducción a la
Programación

Funciones

- Alcance de una variable
 - La definición o redefinición de una variable dentro de una función tiene un alcance local

```
>>> a = 100  
>>> def sumaValorA(x):  
    return x + a  
>>> sumaValorA(1)  
101
```

CC1002 Introducción a la
Programación

Funciones

- Alcance de una variable
 - La definición o redefinición de una variable dentro de una función tiene un alcance local

```
>>> a = 100
>>> def sumaValorA(x):
        a = 200
        return x + a
>>> sumaValorA(1)
201
>>> a
100
```

CC1002 Introducción a la
Programación

Funciones

- Alcance de una variable
 - La definición o redefinición de una variable dentro de una función tiene un alcance local

```
>>> a = 100
>>> def sumaValorA(a):
        return 1 + a
>>> sumaValorA(5)
6
```

CC1002 Introducción a la
Programación

Error de ejecución

- El error se detecta cuando se realiza la evaluación de una expresión

```
>>> 1 / 0
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    1 / 0
ZeroDivisionError: division by zero
```

CC1002 Introducción a la
Programación

Error de ejecución

- El error se detecta cuando se realiza la evaluación de una expresión

```
>>> areaCirculo(5,3)
Traceback (most recent call last):
  File "<pyshell#1>", line 1, in <module>
    areaCirculo(5, 3)
TypeError: areaCirculo() takes 1 positional argument but 2 were given
```

CC1002 Introducción a la
Programación

Receta de diseño de funciones

- El desarrollo de una función requiere varios pasos. Necesitamos saber:
 - qué es lo relevante en el enunciado del problema y que podemos ignorar
 - qué es lo que la función recibirá cómo parámetros, y como los relaciona con la salida esperada

CC1002 Introducción a la
Programación

Receta de diseño de funciones

- El desarrollo de una función requiere varios pasos. Necesitamos saber:
 - si Python provee operaciones básicas para manejar la información que necesitamos trabajar en la función
 - una vez que tengamos desarrollada la función, necesitamos verificar si efectivamente realiza el calculo esperado (para el cual efectivamente implementamos la función)

CC1002 Introducción a la
Programación

Receta de diseño de funciones

- Receta de diseño:
 - Entender el propósito de la función
 - Dar ejemplos de uso de la función
 - Probar la función (testing)
 - Finalmente, especificar el cuerpo de la función

CC1002 Introducción a la
Programación

Entender el propósito de la función

- Ejemplo: función `areaAnillo`
 - Empezar cada función dándole un nombre significativo, especificando que tipo de información consume y que tipo de información produce
 - A esto lo llamamos **contrato**

```
# areaAnillo: num num -> float
```

CC1002 Introducción a la
Programación

Entender el propósito de la función

- Ejemplo: función `areaAnillo`
 - El tipo de datos que consume es de tipo numérico (**int** o **float**), por lo que lo representaremos con la palabra **num**
 - El valor que produce sólo puede ser de tipo real (**float**)
 - Luego, agregamos el encabezado

```
# areaAnillo: num num -> float
def areaAnillo(exterior, interior):
```

CC1002 Introducción a la
Programación

Entender el propósito de la función

- Ejemplo: función `areaAnillo`
 - Explicitamos el propósito de la función en un comentario luego del contrato (todas las líneas que sean necesarias)

```
# areaAnillo: num num -> float
# Calcula el area de un anillo de radio exterior
# y cuyo agujero es de radio interior
def areaAnillo(exterior, interior):
```

CC1002 Introducción a la
Programación

Dar ejemplos de uso de la función

- Ejemplo: función `areaAnillo`
 - Evaluar ejemplos y calcular manualmente
 - la función `areaAnillo` debe generar el valor 50.24 para las entradas 5 y 3

```
# areaAnillo: num num -> float
# calcula el area de un anillo de radio exterior
# y cuyo agujero es de radio interior
# Ejemplo: areaAnillo(5, 3) devuelve 50.24
def areaAnillo(exterior, interior):
```

CC1002 Introducción a la
Programación

Probar la función (Testing)

- Ejemplo: función `areaAnillo`
 - Asegurar que la función calcula el valor esperado al menos para los ejemplos definidos

```
# areaAnillo: num num -> float
# Calcula el area de un anillo de radio exterior
# y cuyo agujero es de radio interior
# Ejemplo: areaAnillo(5, 3) devuelve 50.24
def areaAnillo(exterior, interior):
    ...
```

```
# Tests
assert areaAnillo(5, 3) == 50.24
```

CC1002 Introducción a la
Programación

Implementar el cuerpo de la función

- Ejemplo: función `areaAnillo`
 - Finalmente, escribir el código de la función

```
# areaAnillo: num num -> float
# Calcula el area de un anillo de radio exterior
# y cuyo agujero es de radio interior
# Ejemplo: areaAnillo(5, 3) devuelve 50.24
def areaAnillo(exterior, interior):
    return areaCirculo(exterior) - areaCirculo(interior)

# Tests
assert areaAnillo(5, 3) == 50.24
```

CC1002 Introducción a la
Programación

Probar la función

- Ejemplo: función `areaAnillo`

```
>>> assert areaAnillo(5, 3) == 50.24
>>> assert areaAnillo(5, 3) == 0
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AssertionError
```
- Nota: más adelante veremos que preguntar por la igualdad de dos números reales puede traer problemas (en este ejemplo funciona)

CC1002 Introducción a la
Programación

Receta de diseño de funciones

- Resumen: pasos para cumplir con la receta de diseño:
 - Escribir el contrato de la función
 - Escribir el propósito de la función
 - Escribir ejemplos de uso de la función
 - Implementar tests para la función
 - Implementar la función

CC1002 Introducción a la
Programación

Para la próxima clase

- Leer el Capítulo 4 del apunte

CC1002 Introducción a la
Programación